Weeks 7-13

Logbook 7

• Notes from at least two presentations by other teams.
• One technique or approach you saw that you would like to try.
• A reflection on your own submission: what went well and what you would improve.

Presentation 1 (The Williams):

- The Williams have all stats placed in one single window
- Troops move along the grid automatically
- Simulation has multiple rounds
- Round ends if their troops dies or other team's castle get destroyed
- Included are exceptions
- Different types of troops in their simulation (ranged, etc)
- They have numerous 'game states' in their simulation.
- One thing i would like to try in our project is the implementation of throws and exceptions.
- one thing i liked from this presentation is their ability to implement all the logic on one window, our one has a separate window for stats as opposed to having it all in one window.

Presentation 2 (Team B):

- Tile based grid
- Terrain on different tiles
- Natural disasters can occur during simulation
- Grid size can be manipulated over time --> this is something i like and would like to try and implement in our project.
- Two different nations that try and take over the board over time.

Presentation 3 (Starfish):

Presentation 4 (Table):

- A simulation of a planet's gravity.
- This one is my favorite one.

Logbook 8

some best zingers from today's debates:

- 'computationally inefficent for multiple child observers to run timers'

When we were debating team oops x table i thought a really good argument they bought up was scalability, they argued that enums were not good for scalability.

Logbook 9

We didn't have a class during this week as it was a public holiday.

Logbook 10

Today we refreshed our knowledge on streams and lambdas. Below is a piece of code in our window class that we could potentially update to minimize the amount of lines and make our program cleaner.

```
//loop for vertical lines
for (int i = 0; i < 100; i++) {
    g2d.drawLine(i * 50, y1: 50, i * 50, y2: 550);
}

//loop for horizontal line
for (int i = 0; i < 100; i++) {
   g2d.drawLine(x1: 50, i*50, x2: 750, i*50);
}
```

Logbook 11

Starfish was able to group their states (infected, healthy, dead, etc) to display the amount of each are in each cell. Collectors were implemented to group these states together and gather information relating to their population.

Our group decided to implement a collector to gather the total complete population of our city's simulation. This was done through a summarizingDouble collector which allows us to use a getSum method to achieve what we want to do with a collector.

Logbook 12

Team B has identified a key area to place parallelism, which is in their nation logic. From there they will implement a stream to divide up their land tiles into lists and concurrently sort through the lists to save memory/time, this divides the workload twofolds, one stream responsible for another set of tiles and another for another set.

This was my favorite implementation of parallelism as it served an actual purpose within their program and actually helped optimize performance.

We decided to implement parallelism by dividing our grid into 4 sections/streams to run the placement logic better and efficiently.

Logbook 13

One of the presentations, The William’s battle simulation program. Their entire interface looks really well built, they have also chosen to include custom collectors inside their program to keep track of troop health.